

4. 필터 구현용 베이스 클래스와 등록 방법

4.1 GenericFilterBean : 필터를 Spring Bean 만 들기

GenericFilterBean : 스프링 빈으로 구성되는 필터를 위한 베이스 클래스.
핵심 특징으로는 SpringApplicationContext 생명주기와의 통합이다.

왜 중요한가?

순수 Servlet Filter 는 Servlet Container 가 생성하고 관리한다.
그 안에서 Spring Bean 을 주입받기가 자연스럽지 않다.

하지만 **GenericFilterBean** 을 상속하면 필터 자체가 Spring Bean 이 되며 다음과 같은 특징을 가질 수 있다.

- 의존성을 주입 받을 수 있다.
- 초기화, 소멸 시점이 `Spring Context` 의 생명주기에 맞춰 관리 가능

즉, 서블릿 스펙의 필터와 Spring 의 DI 를 연결할 수 있다.

4.2 OncePerRequestFilter : 요청당 정확히 한 번만 실행

GenericFilterBean 을 확장한 클래스

요청 시작 시점에 단 한번의 호출만 지원한다

구체적으로는 `REQUEST` 디스패치 단계에서 실행되며 `FORWARD` 디스패치를 통한 추가 처리는 무시된다. 서블릿 컨테이너에서는 하나의 클라이언트 요청이 내부적으로 여러 번 디스패치 될 수 있다.

예를 들어 컨트롤러가 뷰로 forward 하면 필터 체인이 다시 터질 수 있는데 그러면 로깅이나 인증로직이 중복 실행될 수 있는 문제점이 발생한다. **OncePerRequestFilter**는 이를 방지해 하나의 요청은 한 번의 실행을 보장한다.

ASTNC 및 ERROR 디스패치에 관여할 지 여부를 제어

비동기 처리나 에러 처리 과정에서 필터가 다시 개입해야 하는지를 상황에 맞게 결정할 수 있다.

Spring Bean 의 이점을 그대로 물려 받는다.

GenericFilterBean의 확장이므로 스프링 빈의 이점 그대로 물려 받는다.

인증 토큰 검사, 요청 로깅, MDC 처럼 요청 당 한 번만 확실히 실행되어야하는 로직은 OncePerRequestFilter 를 상속한다.

4.3 Filter 를 등록하는 법

공식문서에서는 서블릿 필터를 설정하는 2가지 방법을 언급한다.

- web.xml
- Servlet Annotation

전통적인 Spring MVC 환경에서는

AbstractDispatcherServletInitializer 가

편리한 방법을 제공한다. `getServletFilter()` 을 오버라이딩해서 Filter 배열을 반환한다.

각 필터가 구체 타입에 기반한 기본 이름으로 추가되고
DispatcherServlet에 자동으로 매핑

```
public class MyFilter extends
AbstractDispatcherServletInitializer{

//.. 생략 ..

@Override
protected Filter[] getServletFilters() {
    return new Filter[] {
        new CharacterEncodingFilter("UTF-8", true),
        new HiddenHttpMethodFilter()
    };
}

//..생략..
}
```

또한 `isAsyncSupported()` 라는 `protected` 메소드가 있어서
DispatcherServlet과 그것에 매핑된 모든 필터의 비동기 자원을 한 곳에
서 켤 수 있으며 기본 값은 `true` 이다.

필터 순서는 배열 순서 = 등록 순서이다. `CharacterEncodingFilter` 는
요청 본문을 읽기 **전에** 인코딩을 세팅해야 하므
로 `HiddenHttpMethodFilter` (본문의 `_method` 파라미터를 읽음)보다
앞에 와야 합니다. 지금 코드 순서는 뒤집혀 있습니다.

클래스명: `MyFilter` 인데 실제로는 필터가 아니
라 `WebApplicationInitializer` 구현체입니다. 여기 등록되는 필터들
은 DispatcherServlet에만 매핑되고(`ROOT` 매핑이 아
님), `getServletMappings()` 가 반환한 경로로 자동 매핑됩니다.

그리고 현실적인 얘기 하

나 — `AbstractDispatcherServletInitializer` 는 WAR 배포 용 `web.xml` 대체재입니다. Spring Boot 3.x 임베디드 톰캣 환경이라면 이 클래스는 아예 쓰지 않습니다. Boot에서는 필터를 `@Bean` 으로 등록하면 `ServletContextInitializer` 가 자동으로 잡아가고, 순서나 URL 패턴을 제어하려면 `FilterRegistrationBean` 을 씁니다:

4.4 등록시 반드시 챙겨야하는 것

필터가 비동기 요청과 에러 디스패치에서도 동작해야한다면 디스패치 타입 매핑을 명시해야한다.

Spring Framework의

`AbstractAnnotationConfigDispatcherServletInitializer` 를 사용하는 경우 모든 디스패치 타입에 자동으로 등록된다.

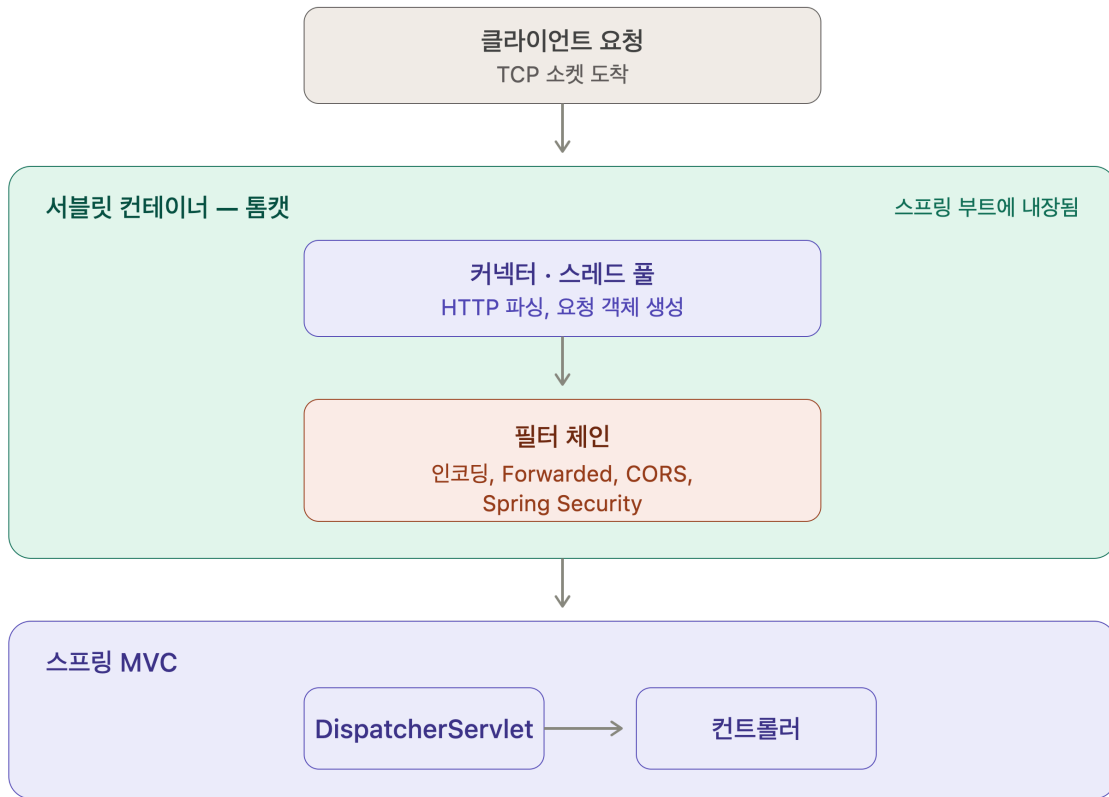
하지만 `web.xml` 로 등록하거나 Spring Boot 에서 `FilterRegistrationBean` 으로 등록하는 경우에는 `DispatcherType.ASYNC`, `DispatcherType.ERROR` 등과 같이 필요한 것을 직접 포함 시켜야한다.

실제로 `ForwardHeaderFilter` 은 `ASYNC`, `ERROR`
`ShalloEtagHeaderFilter` 는 `ASYNC` 가 필요하다.

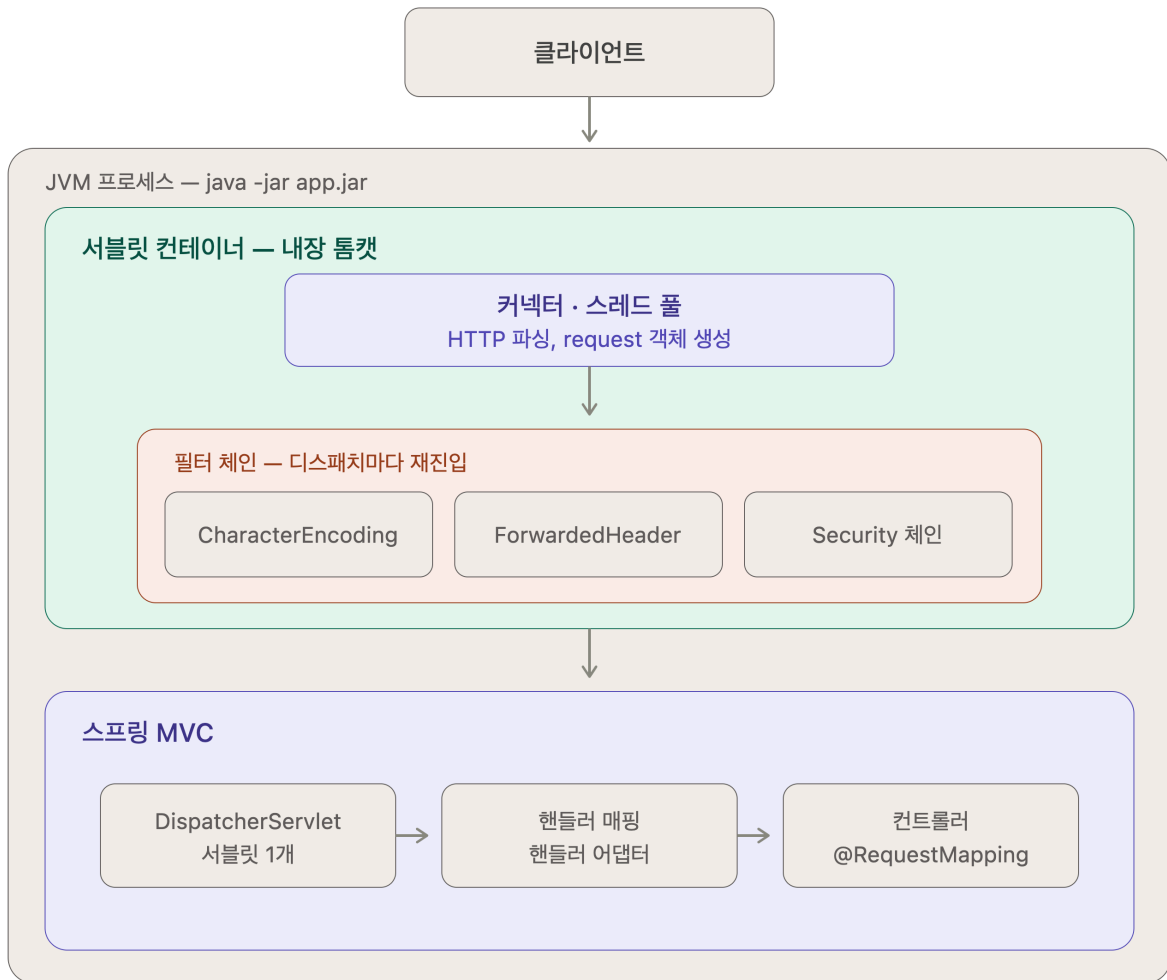
4.5 필터 순서

`ForwardHeaderFilter` 는 요청을 래핑하므로 수정된 요청을 사용해야하는 `RequestContextFilter` 같은 필터보다 앞에 와야하고 `CorsFilter` 은

Spring Security 필터 체인보다 앞에 와야한다.

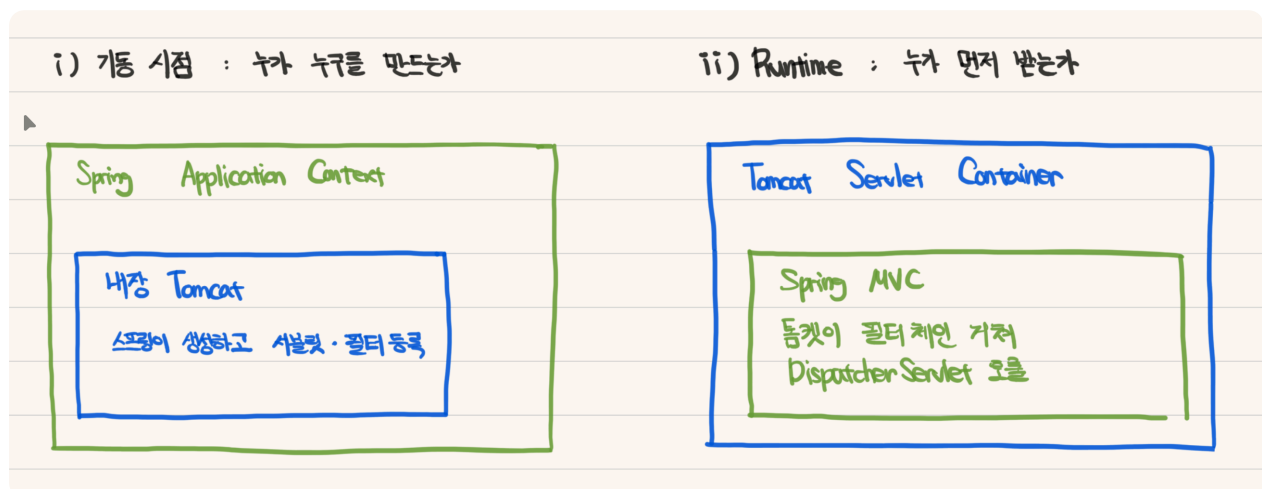


간단하게 정리



여기서 CharacterEncoding, ForwardedHeader, Security 체인이 필터 예시이다. 또한 시큐리티 체인이 DispatcherServlet 보다 바깥에 위치한다.

기동과 런타임에 바깥과 안이 서로 바뀐다 : Tomcat과 Spring



계층별 책임

계층	다루는 것	모르는 것
커넥터	Socket, HTTP 파싱, 스레드 배정	애플리케이션 로직 전부
필터 체인	request / response 객체, 헤더, 인증	어느 컨트롤러가 처리할지
DispatcherServlet	URL <-> 핸들러 매핑, 예외처리	소켓, 스레드 관리
컨트롤러	도메인로직, 응답 데이터	서블릿

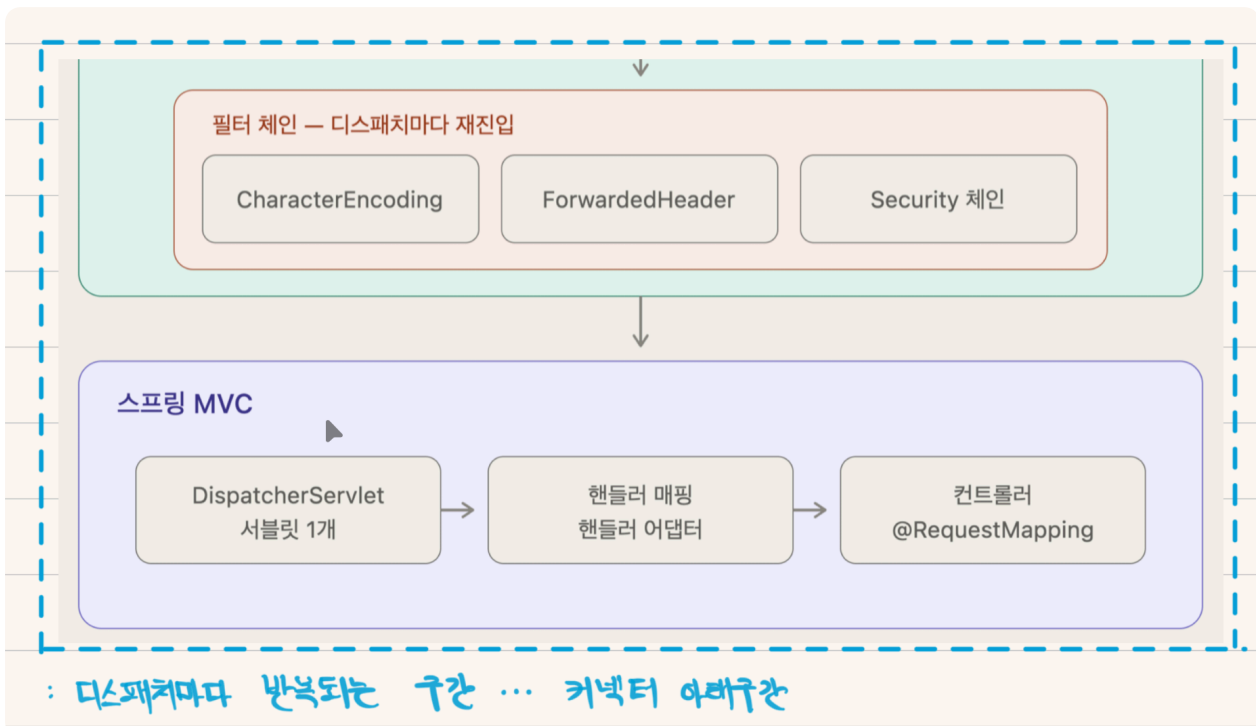
여기서 필터가 컨트롤러를 모른다는 점이 중요하다.

필터는 서블릿 스펙 요소라서 스프링을 몰라도 동작한다.

그래서 ForwardedHeaderFilter 가 하는 일이 낮은 수준에 머무는 것이며 `@RequestMapping` 정보가 필요한 일은 인터셉터나 AOP 의 몫이 된다.

디스패치가 이 그림에 겹쳐지는 방식

HTTP 요청 1건이 디스패치 여러 번 일으킨다.



`DispatcherType` : 디스패치 발생 원인 분류한 값

- `REQUEST` : 클라이언트 요청 도착 - 필터 기본 매핑

- **FORWARD** : 서버 내부 처리 이관
- **INCLUDE** : 다른 리소스 출력 삽입
- **ASYNC** : 비동기 처리 재개
- **ERROR** : 에러 페이지 이동

REQUEST 만 기본인 이유는 나머지는 같은 HTTP 요청의 연장이다.

즉, 인증을 두 번 할 이유가 없다.

반대로 응답 완성 시점에 개입해야하는 필터는 **ASYNC** 명시해야한다.

리다이렉트는 서버 밖으로 나갔다가 새 HTTP 요청으로 들어오므로 새로운 **REQUEST** 디스패치이다.